

Arquitetura comparada

Fábrica de software multi-agente — este trabalho e os sistemas correlatos

Enzo Consulo · 29 de agosto de 2026 · repositório público: github.com/enzoconsulo/Generic-Multi-Agents

Comparação entre quatro sistemas que constroem software com múltiplos agentes. O recorte é arquitetural: como cada um organiza o trabalho, como os agentes se conectam, onde vive o estado, quem decide o que acontece em seguida e como a autoridade de cada papel é imposta. Ferramentas, linguagens e custos ficam de fora de propósito — são consequência das escolhas abaixo, não causa.

1. Os sistemas comparados

- **ChatDev** (arXiv:2307.07924, 2023) — agentes com papéis de empresa de software percorrem um ciclo em cascata. Cada fase é decomposta em subtarefas, e cada subtarefa é resolvida por uma dupla de agentes conversando.
- **MetaGPT** (arXiv:2308.00352, 2023) — mesma metáfora de empresa, mas a coordenação acontece por um quadro de mensagens compartilhado: cada papel publica o que produziu e assina o que lhe interessa. Procedimentos operacionais padrão definem a sequência.
- **softwarefabrik.io** (produto comercial, v0.30.0, 2026) — um assistente coleta o pedido, uma execução isolada produz o trabalho, e um comitê de revisores prepara a decisão final, que é humana.
- **Este trabalho** — o pedido é decomposto numa fila de tarefas nomeadas, com dependências declaradas; cada tarefa percorre um pipeline próprio, e uma máquina de estados decide o que vem depois.

2. A forma de cada arquitetura

Antes das dimensões, a topologia — é ela que explica a maior parte das diferenças seguintes.

Sistema	Como os agentes se conectam
ChatDev	Duplas em diálogo, encadeadas por fase. A conversa é o canal, e o próximo par recebe o resultado da fase anterior.
MetaGPT	Rede desacoplada: ninguém fala com ninguém diretamente. Todos publicam num quadro compartilhado e assinam o que interessa ao seu papel.
softwarefabrik.io	Uma execução linear produz o trabalho; ao fim, um comitê de seis revisores avalia em paralelo e o operador decide.
Este trabalho	Fila de tarefas ordenada por dependência. Cada tarefa percorre construtor, verificador e revisor, em série; tarefas independentes correm em paralelo.

3. Como o trabalho é organizado

Dimensão	ChatDev	MetaGPT	softwarefabrik.io	Este trabalho
Metáfora organizadora	Empresa de software em cascata	Linha de montagem regida por procedimentos	Linha de montagem com posto de inspeção	Linha de produção com fila de tarefas
Unidade de trabalho	Subtarefa de uma fase, resolvida em diálogo	Etapa do procedimento, com artefato de saída	Uma execução sobre o projeto	Tarefa nomeada, com estado, dependências e arquivos declarados
Como o sistema é estendido	Novo papel é um novo texto no elenco fixo	Nova etapa e novo artefato no procedimento	Novo modelo de stack ou novo adaptador	A equipe é sintetizada por projeto; domínio novo exige declarar artefato, geração e verificação

Os três primeiros partem de um catálogo — de papéis, de etapas ou de modelos de stack — definido antes de conhecer o pedido. O quarto não tem catálogo: a especificação, o plano, as tarefas e a equipe são derivados do próprio pedido. A diferença aparece no limite: um catálogo delimita o que o sistema aceita construir; a síntese desloca esse limite para a capacidade de decompor.

4. Como os agentes se conectam, e quem decide o próximo passo

Dimensão	ChatDev	MetaGPT	softwarefabrik.io	Este trabalho
Canal entre agentes	Diálogo direto, em duplas	Quadro de mensagens, com publicação e assinatura	O espaço de trabalho compartilhado	O arquivo da tarefa e o commit
Quem decide o próximo passo	A cascata: a fase seguinte é fixa	O procedimento: a sequência é fixa	O operador humano, no ponto de aprovação	Máquina de estados, a partir das dependências e do estado da tarefa
Lugar do humano na arquitetura	Fornece o pedido e observa	Fornece o pedido e observa	É componente do laço: o portão de aprovação	Fornece o pedido; fica fora do laço de decisão

Aqui está a divergência de fundo. Em ChatDev e MetaGPT a ordem do trabalho é **fixada no desenho** — cascata ou procedimento — e não muda em execução. Em softwarefabrik a ordem existe, mas o avanço depende de uma decisão humana. Neste trabalho a ordem é **derivada em execução**: uma tarefa só fica disponível quando suas dependências concluem, e a fila se reordena sozinha a cada mudança de estado.

5. Onde vivem o estado e a autoridade

Dimensão	ChatDev	MetaGPT	softwarefabrik.io	Este trabalho
Onde vive o estado	Memória de diálogo: curta dentro da fase, longa entre fases	Quadro de mensagens e memória global, consultáveis pelos papéis	Espaço de trabalho versionado, mais o estado de espera por aprovação	Arquivo de tarefa com estado explícito, e banco transacional; a conversa nunca é fonte
Interface entre papéis	A própria conversa	Artefatos estruturados: requisitos, projeto, interfaces	O espaço de trabalho e as diferenças acumuladas	O arquivo da tarefa, com seções separadas por papel
Onde a autoridade do papel é definida	No texto do papel	Na especificação de ação do papel, dentro do procedimento	Em arquivos de papel escritos no espaço de trabalho antes da execução	No catálogo de ferramentas do papel: o revisor não recebe a ferramenta de escrever

A linha do meio é a mais reveladora. MetaGPT resolve a comunicação com **artefatos padronizados** em vez de conversa, o que é a mesma intuição deste trabalho — a diferença é que ali o artefato circula entre papéis dentro de uma execução, e aqui ele é o estado persistente da tarefa, que sobrevive ao fim da sessão.

A última linha é a que separa três abordagens de um mesmo problema. Nos dois sistemas acadêmicos, a autoridade é **texto que o modelo interpreta**; no comercial, é uma **barreira de interface, com o humano atrás dela**; aqui, é a **ausência da ferramenta no catálogo do papel** — o revisor não deixa de corrigir porque foi instruído a não corrigir, e sim porque a ação não existe para ele.

6. Qualidade, fracasso e concorrência

Dimensão	ChatDev	MetaGPT	softwarefabrik.io	Este trabalho
Controle de qualidade	Revisor e testador dentro do diálogo; o revisor aponta e o programador corrige	Revisão embutida nas etapas do procedimento, com artefato conferido	Comitê de seis revisores em paralelo, e decisão humana ao fim	Dois portões em série, com perguntas distintas: funciona? e é o que foi pedido?
Resposta ao fracasso	Limite declarado de rodadas por subtarefa;	Repetição dentro da etapa do procedimento	O operador decide: aprova, rejeita ou	Escada: sobe de modelo, troca de especialista,

	passa adiante ao atingir		intervém	redivide a tarefa e só então bloqueia
Concorrência	Sequencial: uma conversa por vez	Sequencial pelo procedimento, com o quadro permitindo desacoplamento	Uma execução por vez, isolada em contêiner	Até três tarefas em paralelo, quando os arquivos declarados não se cruzam

Os quatro separam quem produz de quem avalia, e três dos quatro limitam a repetição. A diferença na linha do meio não é ter limite: é o que acontece ao atingi-lo. Nos outros, o sistema segue adiante ou entrega o problema ao humano; aqui, a resposta muda de natureza a cada degrau — primeiro muda o modelo, depois muda o agente, depois muda a própria tarefa. O terceiro degrau é o incomum: tratar o fracasso repetido como sintoma de tarefa mal dimensionada, e redividi-la automaticamente.

Na última linha, a fronteira de isolamento também difere. O sistema comercial isola por contêiner, um por execução; aqui o isolamento é lógico, pelos arquivos que cada tarefa declara tocar, verificado antes do despacho — o que permite paralelismo dentro de uma mesma árvore de trabalho.

7. O que as quatro arquiteturas compartilham

A convergência é grande, e reconhecê-la é parte da comparação. Cinco escolhas estruturais aparecem nos quatro sistemas:

- **Papéis especializados no lugar de um agente único.** Todos rejeitam a ideia de um único agente generalista, e todos relatam ganho com a especialização.
- **Decomposição antes da execução.** O pedido nunca é entregue inteiro ao modelo: vira fases, etapas, execuções ou tarefas.
- **Separação entre quem produz e quem avalia.** Em nenhum dos quatro quem escreveu é quem aprova — o que muda é como a separação é imposta.
- **Artefato versionado como saída.** O controle de versão é o meio comum de registrar o que foi produzido.
- **Algum limite de repetição.** Três dos quatro declaram um teto explícito; no quarto, o humano cumpre esse papel.

LEITURA Que quatro sistemas construídos de forma independente cheguem às mesmas cinco escolhas sugere que elas não são preferências de projeto, e sim respostas necessárias às limitações do substrato — modelos de linguagem sem memória entre execuções, produzindo artefatos que precisam ser conferidos por alguém que não os produziu.

8. As três divergências estruturais

Descontado o que é comum, restam três escolhas que separam as arquiteturas. Cada uma tem consequência direta sobre o que o sistema consegue fazer.

8.1 O estado está dentro ou fora da conversa?

Posição	Sistemas	Consequência arquitetural
Dentro	ChatDev — memória de diálogo em dois níveis	A execução é a unidade: ao terminar, não há a que voltar
Externo, em memória	MetaGPT — quadro de mensagens e memória global	Os papéis se desacoplam, mas o estado ainda vive dentro da execução
Fora, persistente	softwarefabrik — espaço de trabalho; este trabalho — arquivo de tarefa e banco	O trabalho sobrevive ao fim da sessão e pode ser retomado por outra execução

8.2 Quem fecha o laço de controle?

Quem fecha	Sistemas	Consequência arquitetural
A estrutura do processo	ChatDev (cascata) e MetaGPT (procedimento)	A ordem é conhecida antes de começar e não muda em execução
O operador humano	softwarefabrik	O sistema avança na velocidade da atenção de uma pessoa
Código que lê estado e decide	Este trabalho	A ordem é derivada em execução, das dependências e do estado de cada tarefa

8.3 Como a autoridade de um papel é imposta?

Mecanismo	Sistemas	Consequência arquitetural
Texto no papel	ChatDev e MetaGPT	A regra é interpretada pelo modelo, e pode ser contornada por ele
Barreira de interface	softwarefabrik	A regra é garantida, ao custo de exigir um humano presente
Catálogo de ferramentas	Este trabalho	A regra vira impossibilidade: a ação não existe para aquele papel

AS TRÊS EM UMA FRASE As arquiteturas divergem em onde guardam o estado, em quem decide o próximo passo e em como impõem a autoridade dos papéis. Este trabalho ocupa a combinação que os outros não ocupam: estado fora da conversa e persistente, laço de controle fechado por código, e autoridade imposta por ausência de ferramenta — o que o coloca no único ponto do espaço em que a separação entre construir e aprovar se mantém sem operador humano presente.

9. Registro do desenvolvimento

As decisões arquiteturais deste trabalho têm data e commit no repositório público, e aparecem em sequência ao longo de seis semanas, cada uma depois do problema que a motivou:

Commit	Data	Decisão arquitetural
8215dbf	01/08	Índice do projeto entregue pronto, no lugar da varredura de arquivos pelo agente
2255150	01/08	Roteamento por domínio: o elenco de agentes deixa de ser único
1a41ca8	02/08	Contexto passa a ser função do papel, e não do projeto
91559b8	02/08	O laço de coordenação sai do modelo e vira código testado
8017c41	02/08	O avanço deixa de depender de julgamento humano
9a1c186	14/08	O verificador passa a declarar o grau de prova de cada critério
e0bffee	23/08	O portão é fechado nos dois caminhos de saída do construtor

Em operação, a primeira versão entregou 86 tarefas concluídas de 89, em três projetos de domínios distintos — um jogo de tabuleiro em versão web multijogador em tempo real; um serviço permanente embarcado no computador de placa única que controla uma impressora 3D; e uma plataforma de dados com camada de análise por modelos de linguagem.

Referências

- QIAN, C. et al. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924, 2023.
- HONG, S. et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023.
- JANDA, M. Agentic Software Factory. softwarefabrik.io, v0.30.0, 2026.
- CUSUMANO, M. Japan's Software Factories. Oxford University Press, 1991.
- Cortex (github.com/itsHabib/cortex) e Sagents (github.com/sagents-ai/sagents) — agentes supervisionados em Elixir/OTP.